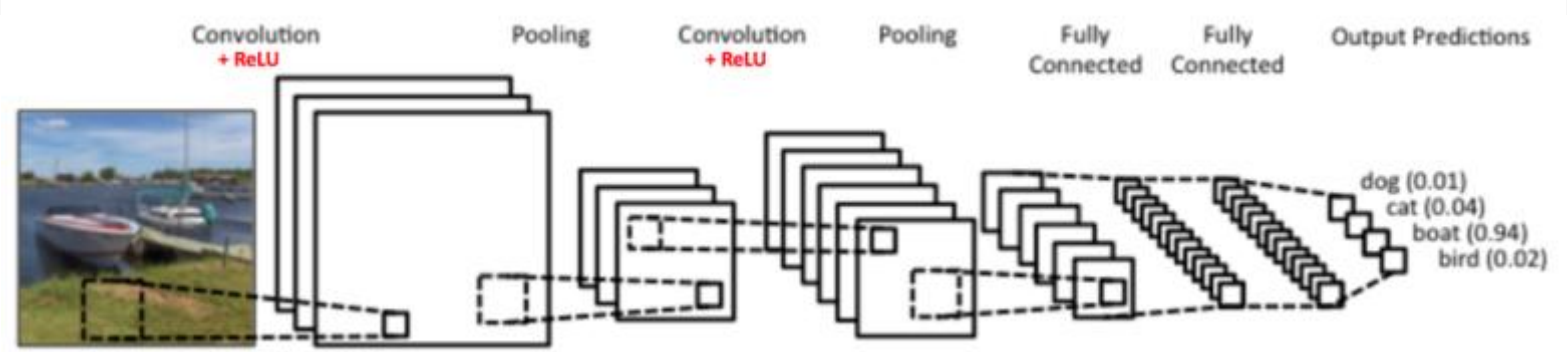
COMP S492F/3920SEF

Convolutional Neural Networks

Convolutional Neural Networks

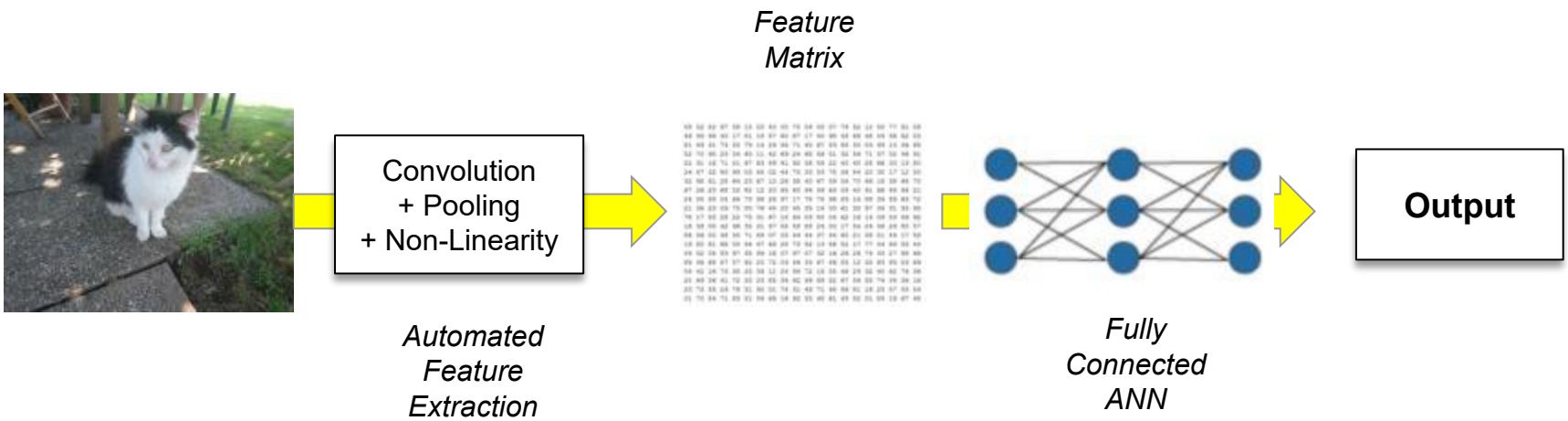
Convolutional Neural Networks (CNN)

- A Convolutional Neural Networks represent a kind of ANN architecture that is composed of four components:
  - Convolution Layers (i.e. an image operation) for Automated Feature Extraction
  - Non-Linearity Activation Function (i.e. ReLU)
  - Pooling or Sub Sampling Layers
  - Fully Connected Layer (i.e. ANN) for Classification
- It is deep learning because there are multiple layers of ANN
- Automated feature extraction for visual data is the main strength
  - There are multiple Convolution Layers for increasingly more sophisticated features



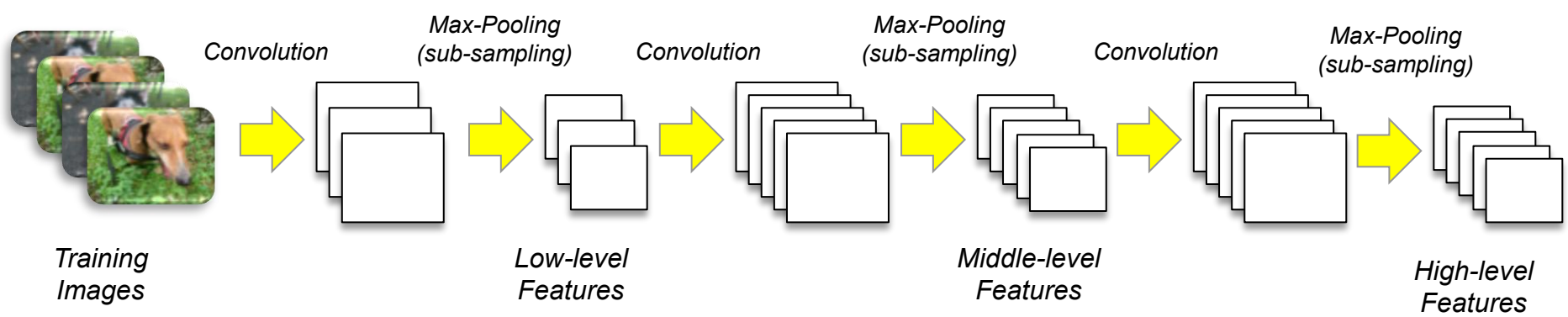
Convolutional Neural Networks (CNN)

- Why so many different layers?
  - Each layer serves some specific purposes
- Two major steps
  - Convolution + Pooling + Non-linearity for extracting features from images
    - There can be many layers of convolution and pooling in alternating manner
  - Fully Connected ANN uses the features as input to train a classifier
- Why convolution for visual data?
  - Because visual features are usually localized



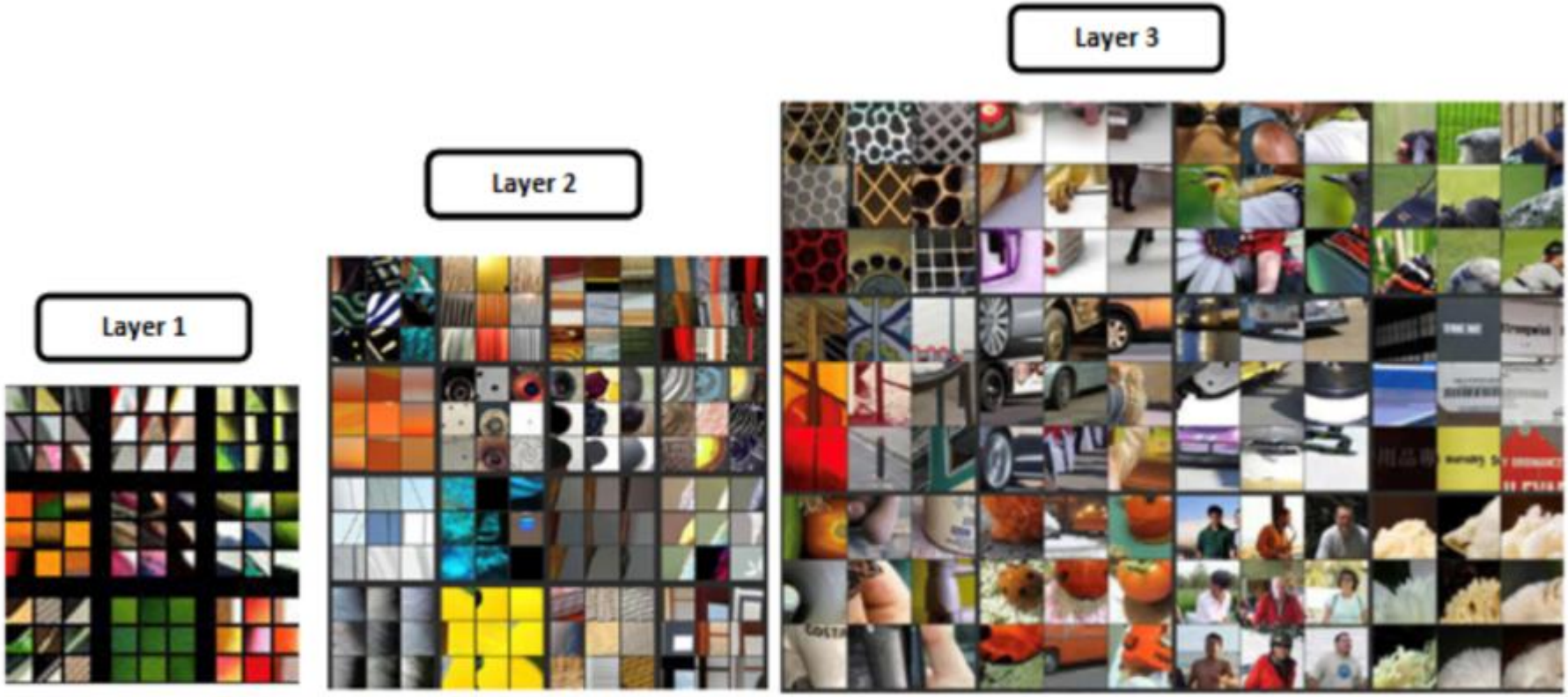
Automated Feature Extraction

- Let's focus on the Convolution part which is for automated feature extraction
  - The convolution layers are for detecting features
    - Pre-defined kernels for detecting vertical lines, horizontal lines, etc
  - The pooling layers are for reduce data dimension (sub-sampling) and for generalize the feature (features shifting a few pixels are not important)
- Alternatively applying convolution and pooling will turn images into low-level features, and from low-level features to high-level features



Automated Feature Extraction

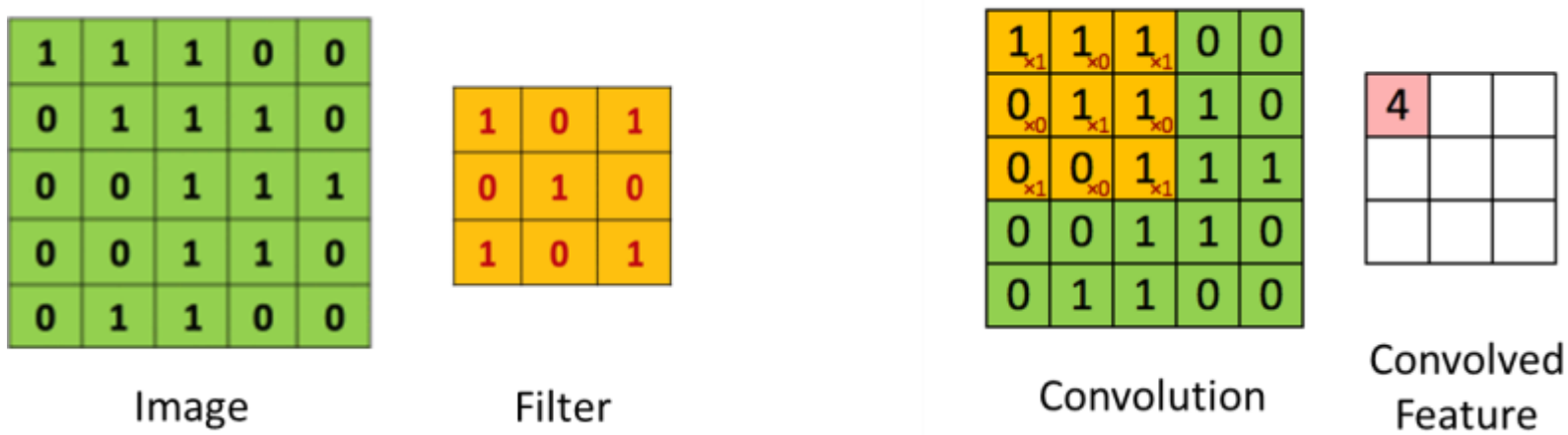
- The following example illustrates a trained CNN and how the feature maps of different layers look like



Feature Visualization of Convolutional net trained on ImageNet (from Matthew D. Zeiler and Rob Fergus)

The Convolution Operation

- Convolution is an operation involves an image and a kernel (i.e. filter) and produces another image
  - Image * Kernel >>> New Image
  - Different kernels (also called filters or feature detectors) produces different images
    - Emphasizes different features found in images
  - The matrix formed is by sliding the filter over the 5x5 image and computing their dot products. It is called the ‘Convolved Feature’ or ‘Activation Map’ or the ‘Feature Map‘



Convolution Filters Examples

- An edge emphasis kernel
  - Note that the edge pixels in the convolved features cannot be calculated because the filter needs a 3x3 sub-image centered on the yellow pixel

10	10	10	10
10	10	10	10
200	200	200	200
200	200	200	200

Image

*

0	-1	0
-1	4	-1
0	-1	0

Filter or Kernel

10	10	10	10
	-190	-190	
	190	190	
200	200	200	200

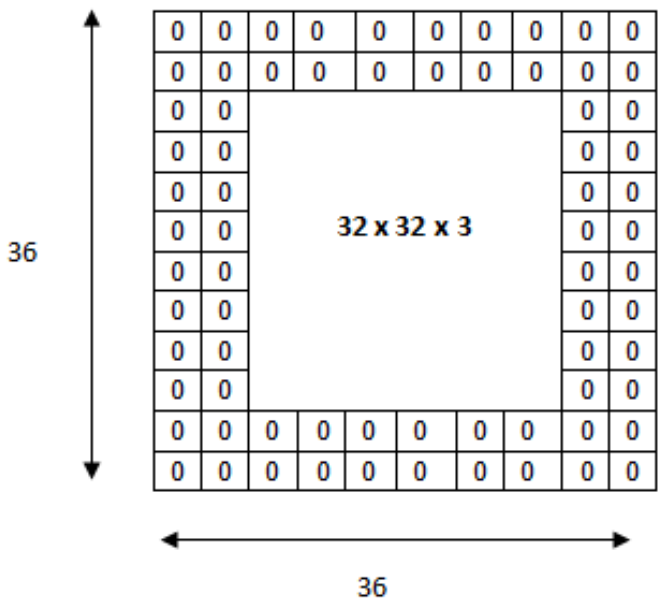
Convolved Features

*= 10 * 4 + 10 * (-1) + 10 * (-1) + 10 * (-1) + 50 * (-1) = -40*

Convolution Filters: Padding

- The convolution operations cannot apply to the pixels right on the sides
  - There are no 3x3 image data at those positions
- Padding the image along the sides before convolution
  - Padding with zeros
  - Padding with side pixels' values

10	10	10	10
10	10	10	10
200	200	200	200
200	200	200	200

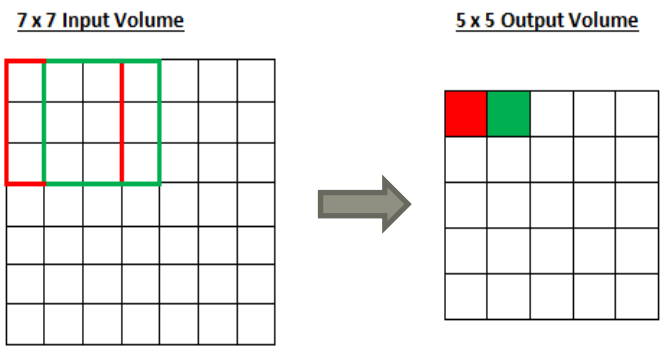


10	10	200	200
10	10	200	200
10	10	200	200
10	10	200	200

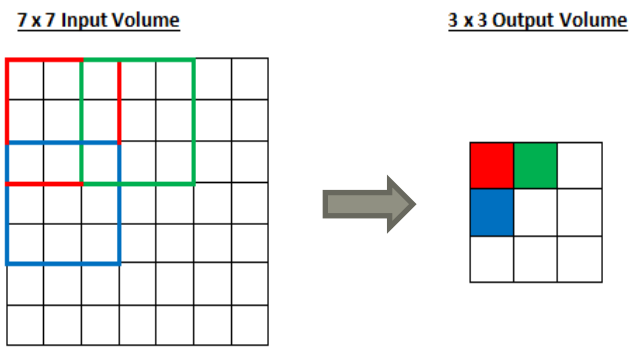
10	10	10		
10	10	10	200	200
10	10	10	200	200
	10	10	200	200
	10	10	200	200

Convolution Stride

- Convolution operation will apply on every pixel, one by one.
- But it can also apply to one pixel and skip another one.
  - The skip length is called the stride
  - Some features are larger in size
    - Skipping pixels can still detect the feature but faster
  - Set in a way so that the output volume is an integer and not a fraction



Stride = 1



Stride = 2

Size of Feature Map

Given

- Image size = *n* x *n*
- Filter size = *f* x *f*
- Padding = *p*
- Stride = *s*

The size of featured map after convolution

$$= \left[\frac{n+2p-f}{s}+1\right] \times \left[\frac{n+2p-f}{s}+1\right]$$

Ex. *n*=32 *f*=5 *p*=1 *s*=1 >>>> size of featured map = 30 x 30

Convolution Filter Learning

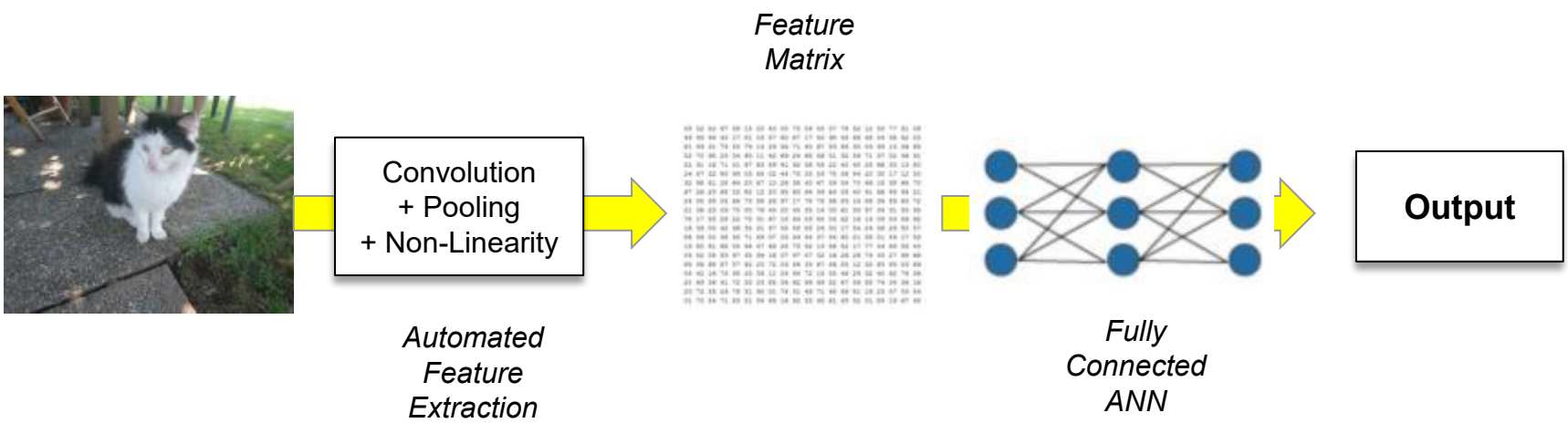
- These filters are learned during training in CNN
  - A set of filters are initialized with random numbers
  - The values in the filters are nudged to the direction to produce greater responses at all places in the images
    - Example: if an image has a lot of vertical edges, then at least one of the filters should converge to one similar to the Sobel Filter.
  - Usually the learned filters represent specific features found in a set of images
- See this example of visualization of learned CNN convolution filters
  - http://deeplizard.com/learn/video/cNBBNAxC8I4

Why Convolution is a Useful “Study Skill” for CNN?

- Images are made of pixels, of which the nearby pixels are highly correlated
  - The surrounding of a yellow pixel is likely to be yellow
  - Any deviation is an anomaly that is potentially a feature
  - Feature can be identified locally
- Convolution can exploit this locality feature of images, learn the kernels, and build features
- Feature visualization allows us to visualize what has been learned in the filters or kernels

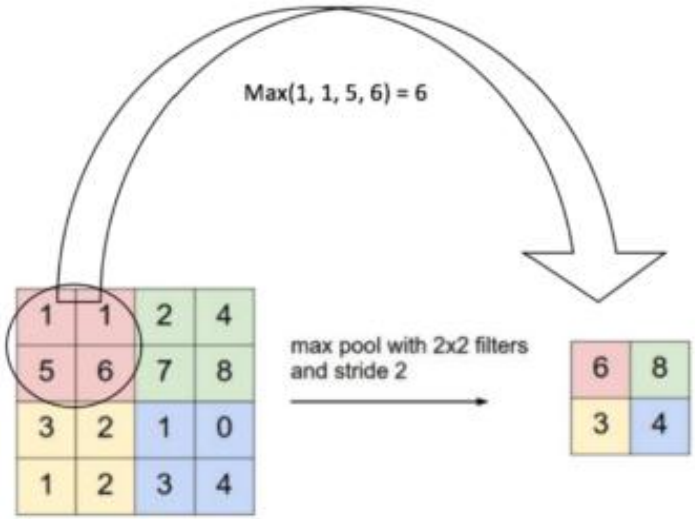
The Pooling Layers

- Feature maps are large
- A 32x32 image with a 5x5 filter (kernel), padding = 1 and stride = 1
  - Size of feature map = 30 x 30
- Many features, many input nodes, ANN very large and deep
  - Training is not efficient
  - Not all the features are needed
  - Some features may be removed with minimal loss
    - Features that are inactivated can be removed
    - Features can be considered collectively



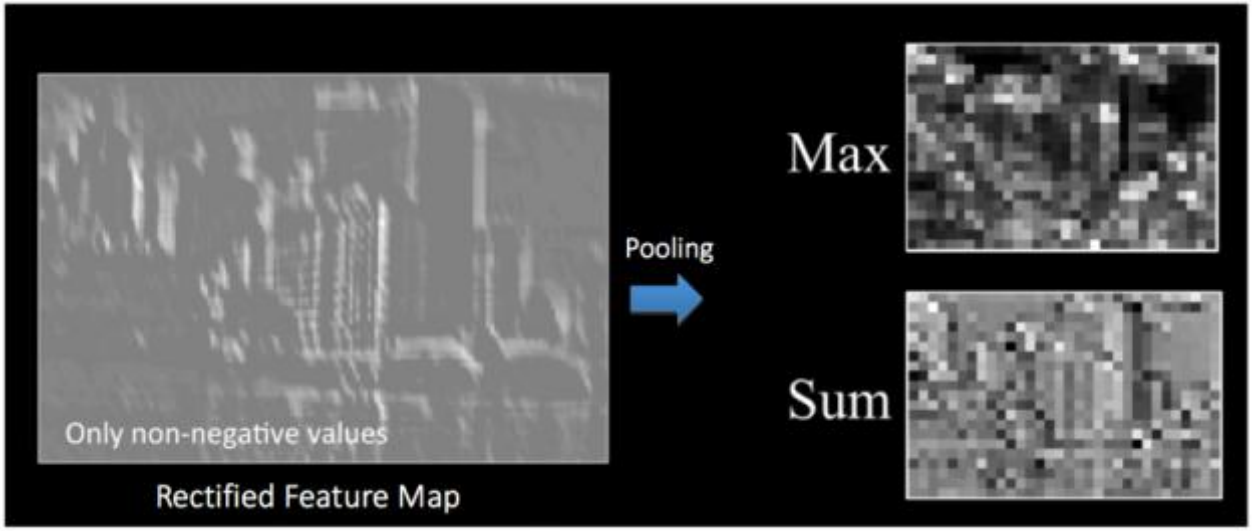
The Pooling Layers

- Spatial Pooling (also called *subsampling* or *downsampling*) reduces the dimensionality of each feature map but retains the most important information.
- It can be of different types: Max, Average, Sum etc.
- Pooling is done by sliding, in this case, a 2 x 2 window by 2 cells (also called '*stride*') and take the maximum value in each region.
  - Remember usually a high value means "activated" by filter
  - Max-pooling: sensitive to any strong feature activation in the region
  - Average-pooling: measure the strength of overall feature activation in the region



The Pooling Layers

- Reduce the size of the input (feature dimension)
- Reduces the number of parameters and computations in the network, therefore, controlling overfitting.
- Makes the network invariant to small transformations, distortions and translations in the input image
  - Almost scale invariant representation of our image (the exact term is “equivariant”). The objects can be detected in an image no matter where they are located.



CNN Model Training

CNN Model Training

- Step 1: We initialize all filters having parameters/weights with random values.
- Step 2: The network takes a training image as input, goes through the forward propagation step (convolution, ReLU and pooling operations along with forward propagation in the Fully Connected layer) and finds the output probabilities for each class.
- Step 3: Calculate the total error at the output layer (summation over all the classes)
  - Total Error = $\sum$ ½ (target probability – output probability) ^{2}
- Step 4: Use Backpropagation to calculate the gradients of the error with respect to all weights in the network and use gradient descent to update all filter values/weights and parameter values to minimize the output error.
  - Note that only the values of the filters and connection weights are updated in the training process.
- Step 5: Repeat steps 2-4 with all images in the training set.

Layer Interleaving in Very Large-Scale CNN

- Very large-scale CNN uses interleaving to setup different types of layers for better expressive power
  - Expressive power refers to the range of functions that a network is able to learn and compute
- CNN uses three types of layers between input and output layers
  - The convolutional layer (C)
  - The pooling layer (P)
  - The activation layer (A) (such as ReLU)
- Some typical patterns
  - CACAP, CACACACACAP
  - CACAPCACAPCACAPCACAPF
  - Fully Connected Layer (F) is also often used
- In very deep CNN, a layer group may be bypassed by another connection known as skip connections

Fully Connected Layer

- A fully Connected Layer, often called FC, is simply a feed-forward network
  - Densely connected
- The purpose of FC is to convert possible multi-dimensional data from earlier layers into a single vector
  - The next layer requires a single vector
- FC is mainly used at the input layer, the output layer, or the output of an intermediate step
  - At the input, it can be used to reduce noise and increase signal strength of the input data
  - A better embedding or representation of the input data
  - At the output, it is often used for a classification tasks

Problems Happen in CNN Model Training

- Gradient Vanishing
  - Each of the neural network's weights receives an update proportional to the gradient of the error function with respect to the current weight in each iteration of training.
  - Backpropagation computes gradients by the chain rule. This has the effect of multiplying n of these small numbers to compute gradients of the "front" layers in an n-layer network, meaning that the gradient (error signal) decreases exponentially with n while the front layers train very slowly.
  - In the worst case, this may completely stop the neural network from further training.
- Gradient Exploding
  - It is the opposite effect which might result when activation functions are used whose derivatives can take on larger values.
  - Gradient clipping: It is one effective solution used to mitigate the problem. The values of the error gradient are checked against a threshold value and clipped or set to that threshold value if the error gradient exceeds the threshold.

Hyper-Parameters

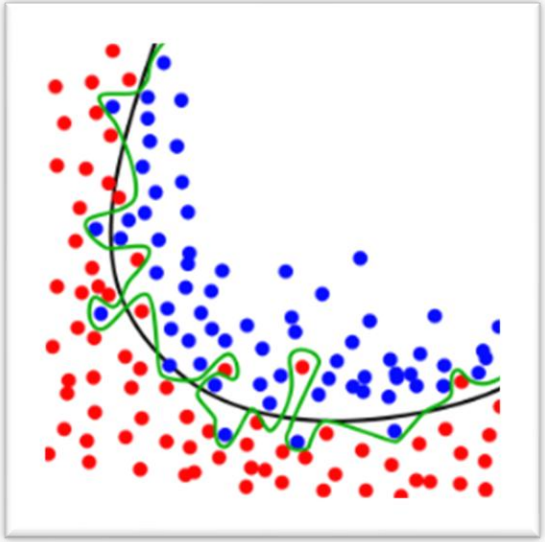
- Hyper-parameters of a learning model (i.e. CNN) are to be determined and set before model training begin. These values are not to be derived from the training data.
  - The number of layers in a CNN and the number of neurons on each layer
  - The learning rate
  - The batch size
    - The number of training examples in one model update pass
    - One pass includes one forward pass and one backward pass in model update
  - The number of epochs
    - One epoch is one complete pass over all training sample
- The hyper-parameters can significantly affect the quality of learning model
- How to set the hyper-parameters?
  - Experience of AI professionals
  - Suggestion of other professionals
  - Grid search
  - Random search

Bias and Variance

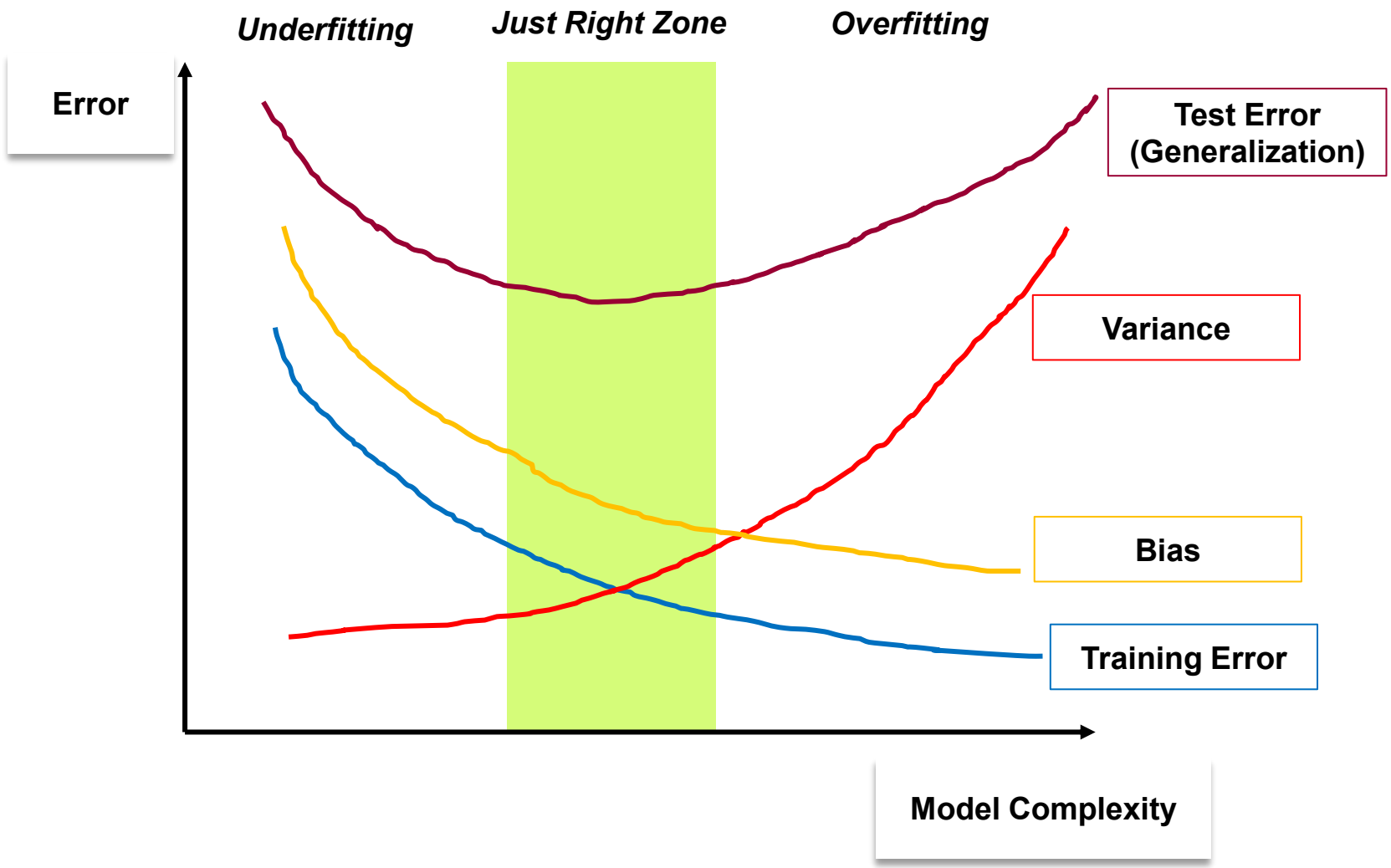
- Should distinguish two types of error in model training
  - Bias is the error on the training dataset
  - Variance is the error on the validation dataset
- High bias is caused by
  - High bias is underfitting
  - Insufficient training epochs
  - Insufficient hidden layers or hidden nodes
  - Unsuitable ANN architecture
- High variance is caused by
  - High variance is overfitting
  - Add more training data
  - Apply regularization technique
  - Unsuitable ANN architecture

Black line
Just right

Green line
Overfitting



Bias and Variance in Model Complexity



Regularization for ANN Training

- Regularization attempts to reduce error in the validation/test dataset by adding an additional penalty term in the loss function
  - The term increases error/loss if the weights are becoming too large
  - The weights are reduced, and the model is simplified
- Techniques
  - L1 Regularization
    - Add absolute magnitude of the weights as a penalty term
    - Tends to shrink the less important features' weights to zero
    - Works well for feature selection
  - L2 Regularization
    - Add squared magnitude of the weights as a penalty term
    - Avoid overfitting
  - Dropout
    - Randomly remove some nodes or weights in the ANN (or set them to zero)
    - The rate of dropout has to be adjusted to strike a balance

Hyper-Parameters: Grid Search

- Grid search
  - A grid of hyper-parameter values is setup
  - For each combination, the hyper-parameter values is used to train a model and evaluate its performance
  - Exhaustive search for the best hyper-parameters
  - Sklearn provides programming support
- Random search
  - A grid of hyper-parameter values is setup
  - The number of hyper-parameter value sets are specified
  - A random set of hyper-parameters values are used and find the best
  - Faster to give something reasonably good but not optimal

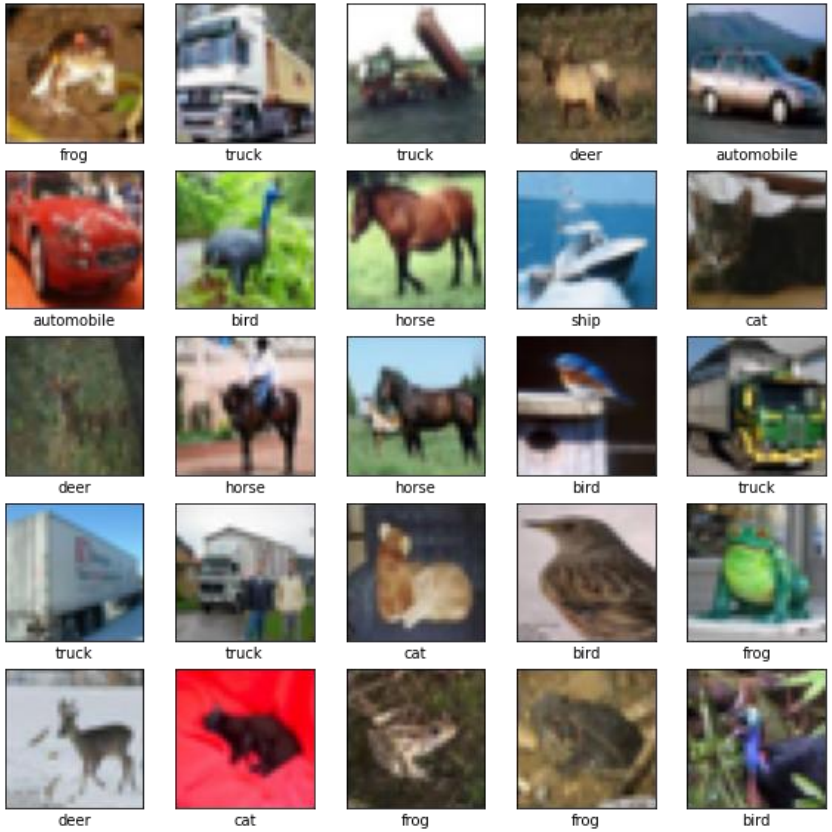
CNN Implementation Examples with Keras

Examples using Tensorflow Keras

- There are many programming tools that help build CNN based deep learning model easily.
- Tensorflow Keras is one of them
  - Tensorflow is the base engine (backend)
  - Keras is the programming interface created for easy programming. Keras may be based on other engines such as Theono.
- Many examples can be found on these online deep learning tutorial pages
  - Google Collaboratory
  - Kaggle
- Without reinventing the wheels, examples from these sites will be used here for illustration.

Example #1: Visual Object Recognition

- Source: https://www.tensorflow.org/tutorials/images/cnn
  - You may run the demo online live with Google Collaboratory
- Classify photos with different objects
  - The CIFAR10 dataset



Example #1: Visual Object Recognition

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt


# DOWNLOAD DATASET

(train_images, train_labels), (test_images, test_labels) =
    datasets.cifar10.load_data()

# Normalize pixel values to be between 0 and 1
train_images, test_images = train_images / 255.0, test_images / 255.0
```

Keras and Tensorflow should have been installed

Normalization is often a good idea in training data to keep the dynamic range of data low

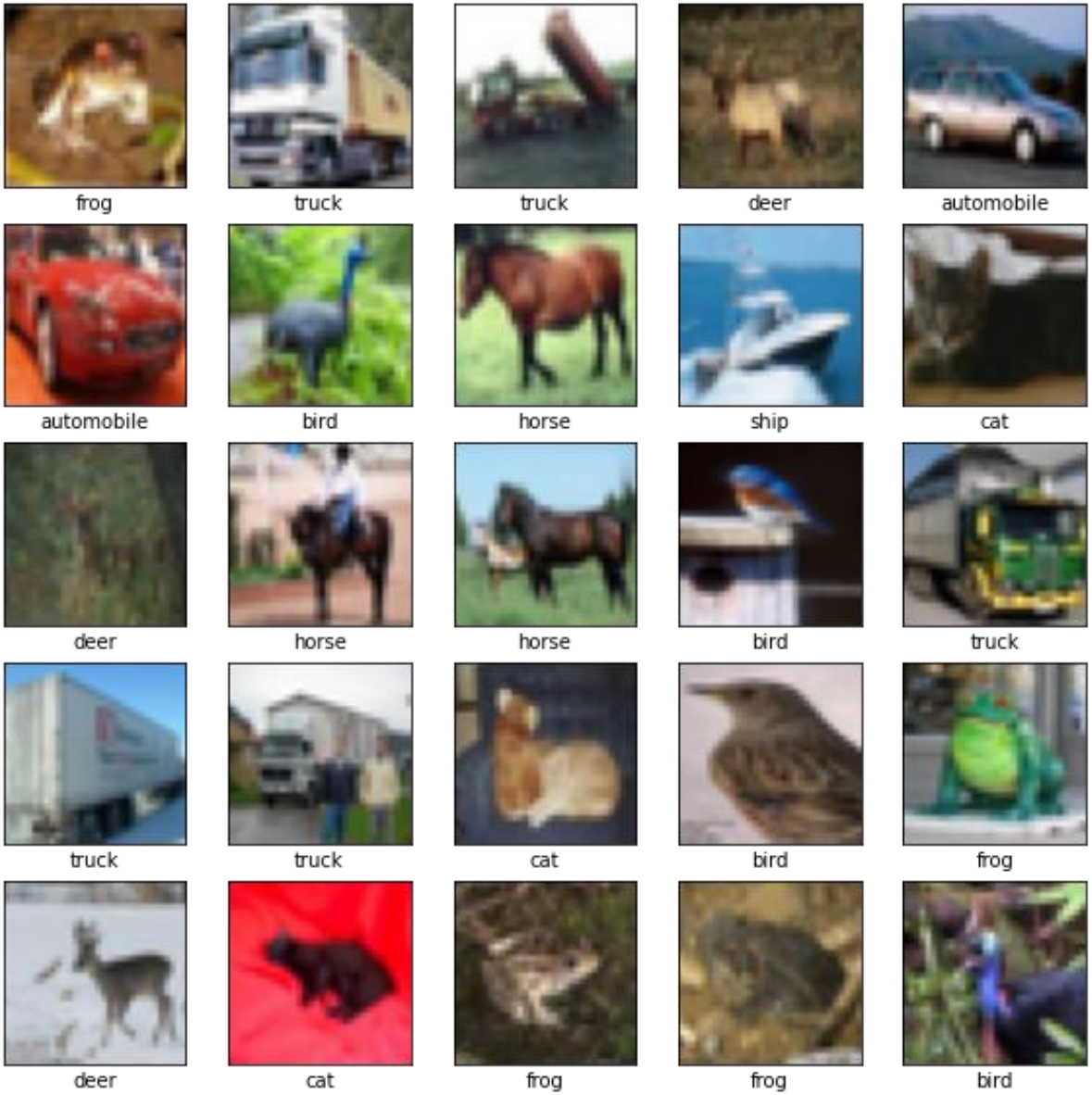
Example #1: Visual Object Recognition

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i], cmap=plt.cm.binary)
    # The CIFAR labels happen to be arrays,
    # which is why you need the extra index
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```

The name of the class has to be hardcoded because the dataset does not contain theses

The dataset is already labeled with an integer indicating the class

Example #1: Visual Object Recognition



Example #1: Visual Object Recognition

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))

model.summary()
```

Building the CNN model. Only the convolutional part is added here

Print the deep model design

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 30, 30, 32)        896
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 15, 15, 32)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 13, 13, 64)        18496
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 6, 6, 64)          0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 4, 4, 64)          36928
=================================================================
Total params: 56,320
Trainable params: 56,320
Non-trainable params: 0
```

Example #1: Visual Object Recognition

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))

model.summary()

model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))
```

Add the classification layer which is a fully connected layer (the Dense layer in Keras)

The output has 10 units
The dataset ground truth label is a one-hot encoded array of 10 classes

Example #1: Visual Object Recognition

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

Adam is a fast optimizer

Compile the model
The loss function is cross-entropy because we want a classification (logistic regression)

```
history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
```

Train the model with 10 epochs

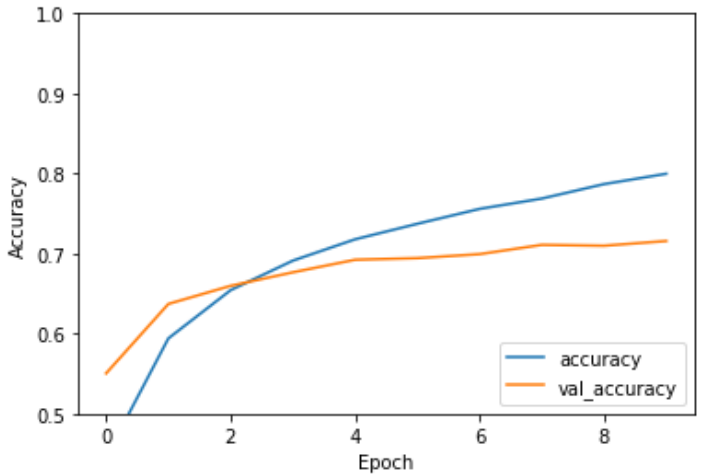
Example #1: Visual Object Recognition

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')



test_loss, test_acc = model.evaluate(test_images,  test_labels, verbose=2)
```

Plot the training accuracy and validation accuracy

Print the results

```
313/313 - 1s - loss: 0.8840 - accuracy: 0.7157
```



Example #2: Flower Recognition

- Source: https://www.tensorflow.org/tutorials/images/classification
- Classify flowers in images
  - Illustrate overfitting and how to resolve the problem with dropout
  - Refer to the above source for the data loading part

Example #2: Flower Recognition

```
num_classes = 5

model = Sequential([
  layers.experimental.preprocessing.Rescaling(1./255, input_shape=(
    img_height, img_width, 3)),
  layers.Conv2D(16, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(32, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(64, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Flatten(),
  layers.Dense(128, activation='relu'),

  layers.Dense(num_classes)
])
```

There are 5 output units for the 5 classes

Example #2: Flower Recognition

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(
              from_logits=True),
              metrics=['accuracy'])

model.summary()



epochs=10

history = model.fit(train_ds, validation_data=val_ds, epochs=epochs)
```

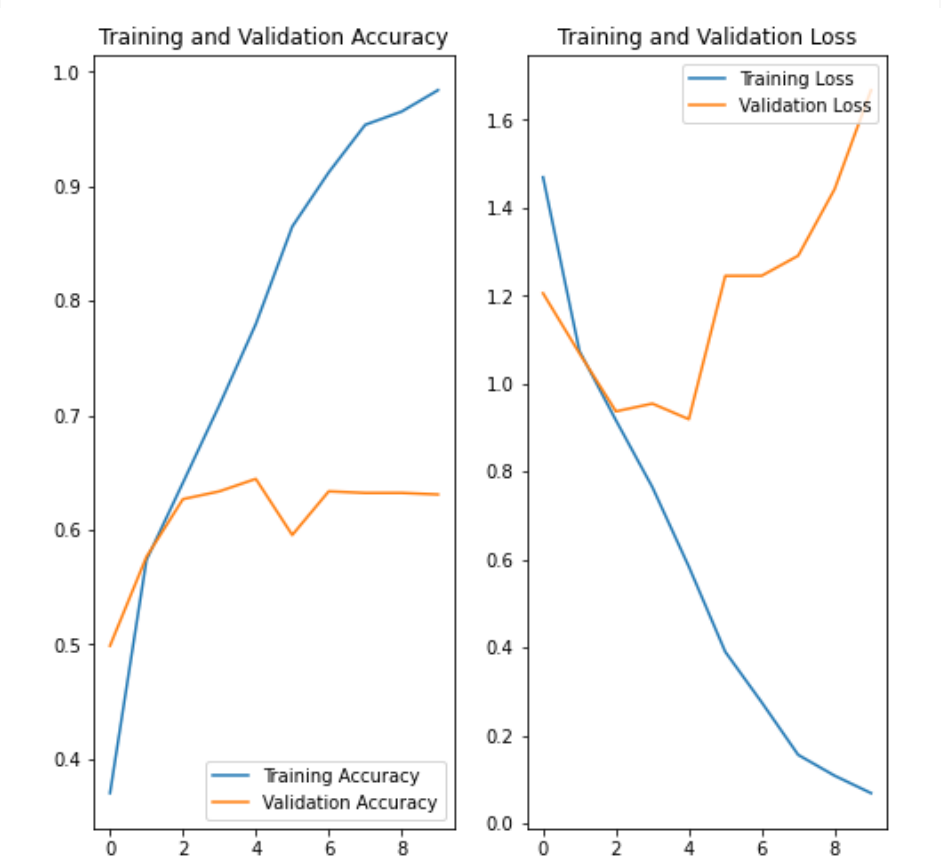
For classification, cross entropy is used as the loss function

Example #2: Flower Recognition

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
rescaling_1 (Rescaling)      (None, 180, 180, 3)       0
_________________________________________________________________
conv2d (Conv2D)              (None, 180, 180, 16)      448
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 90, 90, 16)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 90, 90, 32)        4640
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 45, 45, 32)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 45, 45, 64)        18496
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 22, 22, 64)        0
_________________________________________________________________
flatten (Flatten)            (None, 30976)             0
_________________________________________________________________
dense (Dense)                (None, 128)               3965056
_________________________________________________________________
dense_1 (Dense)              (None, 5)                 645
=================================================================
Total params: 3,989,285
Trainable params: 3,989,285
Non-trainable params: 0
```

Example #2: Flower Recognition

- Plot the training and validation loss
  - The training accuracy is very high but the gap to the validation accuracy is also large
  - Overfit?



Example #2: Flower Recognition

- Two approaches to solve the overfitting problem
  - Approach #1: Data augmentation
  - The number of training sample is not enough (variance too high?)
    - Random transformation of images to create an illusion of different angle of the same flower type
    - The method is domain specific to image recognition

Example #2: Flower Recognition

- Two approaches to solve the overfitting problem
  - Approach #2: Dropout
  - Randomly reduces or eliminate activation of a node (that is dropout)
    - The dropout rate is the proportion of input value used

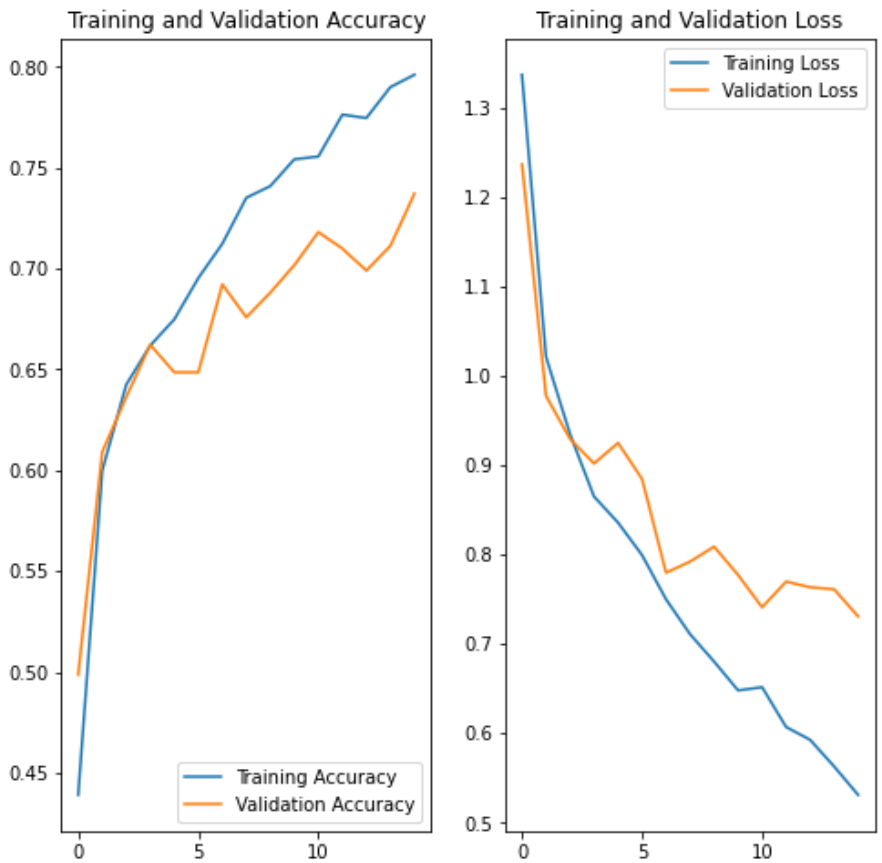
Example #2: Flower Recognition

```
model = Sequential([
  data_augmentation,
  layers.experimental.preprocessing.Rescaling(1./255),
  layers.Conv2D(16, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(32, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(64, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Dropout(0.2),
  layers.Flatten(),
  layers.Dense(128, activation='relu'),
  layers.Dense(num_classes)
])
```

A dropout layer is added here

Example #2: Flower Recognition

- Plot the training and validation loss
  - The validation loss is much reduced and nearer to the training loss



Example #2: Flower Recognition

```
sunflower_url = "https://storage.googleapis.com/download.tensorflow.org/
example_images/592px-Red_sunflower.jpg"
sunflower_path = tf.keras.utils.get_file('Red_sunflower', origin=sunflower_url)

img = keras.preprocessing.image.load_img(
    sunflower_path, target_size=(img_height, img_width)
)
img_array = keras.preprocessing.image.img_to_array(img)
img_array = tf.expand_dims(img_array, 0) # Create a batch

predictions = model.predict(img_array)
score = tf.nn.softmax(predictions[0])

print(
    "This image most likely belongs to {} with a {:.2f} percent confidence."
    .format(class_names[np.argmax(score)], 100 * np.max(score))
)
```

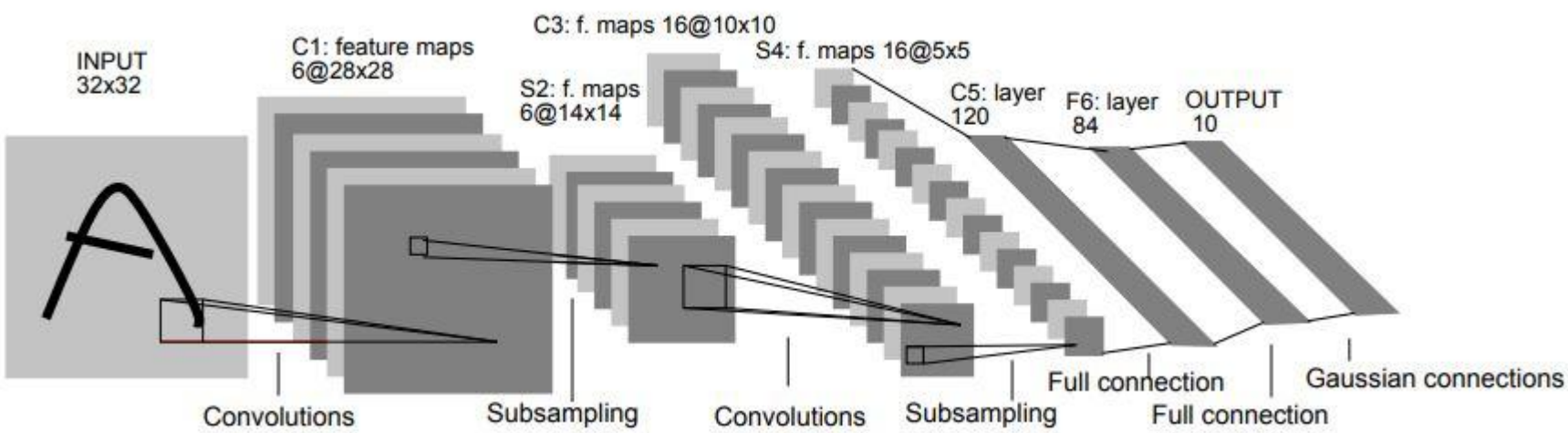
Use the trained model to find the class of an image

```
This image most likely belongs to sunflowers with a 97.38 percent confidence.
```

CNN Renowned Examples

LeNet (LeCun et al. 1998)

- Recognized as the classic CNN architecture
  - Average pooling
  - Sigmoid or tanh nonlinearity
  - Fully connected layers at the end
  - Trained on MNIST digit dataset with 60K training examples
- For Handwriting Recognition (i.e. visual)
- Read about the details: https://engmrk.com/lenet-5-a-classic-cnn-architecture/



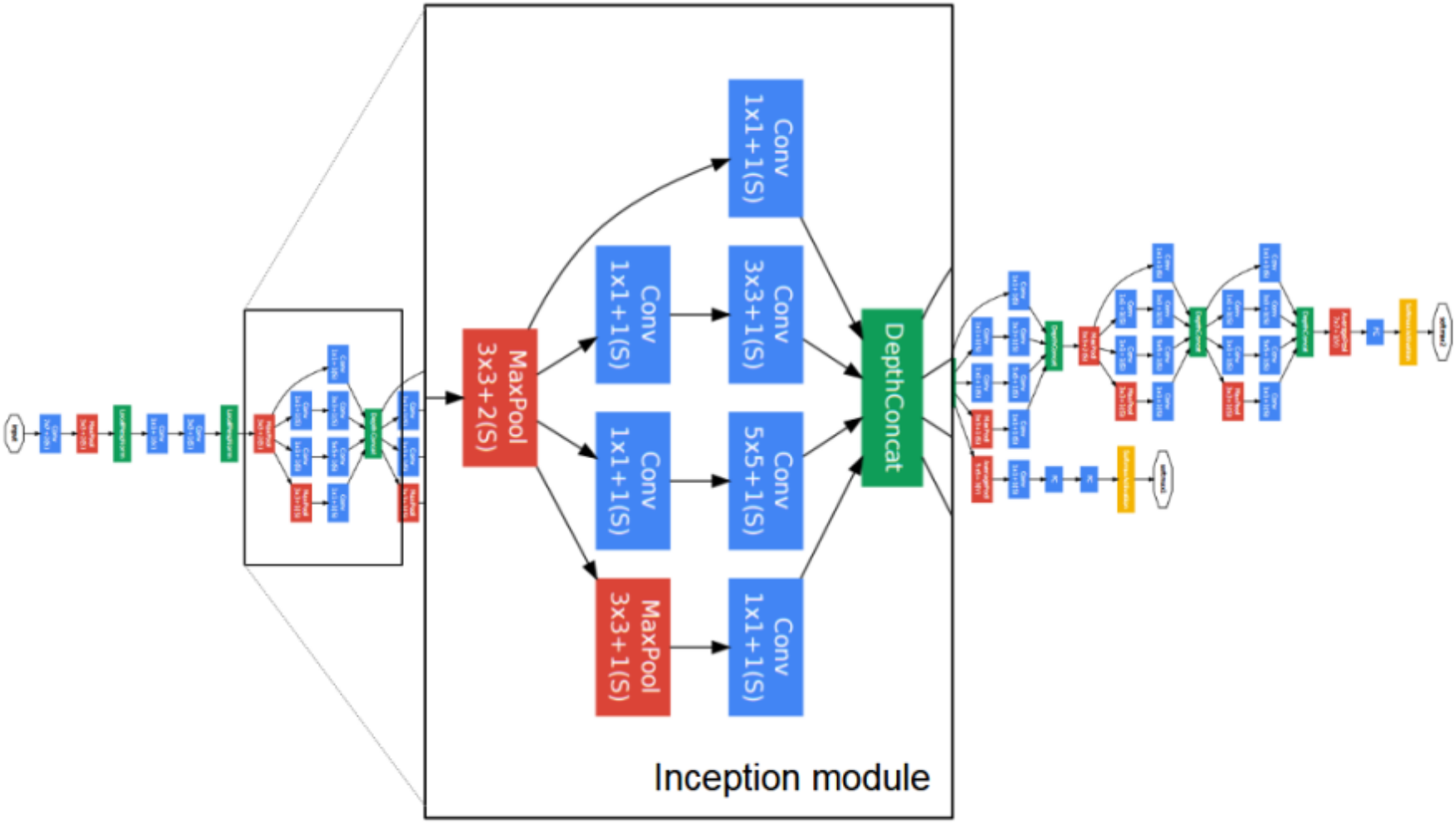
LeNet Architecture Parameters

	Activation shape	Activation size	No. of parameters
Input	(32,32,3)	32x32x3=3,072	0
CONV1 (f=5,s=1)x8	(28,28,8)	28x28x8=6,272	(5x5x3+1)x8=608
POOL1 (f=2,s=2)x8	(14,14,8)	14x14x8=1,568	0
CONV2 (f=5,s=1)x16	(10,10,16)	10x10x16=1,600	(5x5x3+1)x16=1,216
POOL2 (f=5,s=2)x16	(5,5,16)	5x5x16=400	0
FC3	(120,1)	120	(400+1)x120=48,120
FC4	(84,1)	84	(120+1)x84=10,164
Softmax output	(10,1)	10	(84+1)x10=850

GoogLeNet (2015)

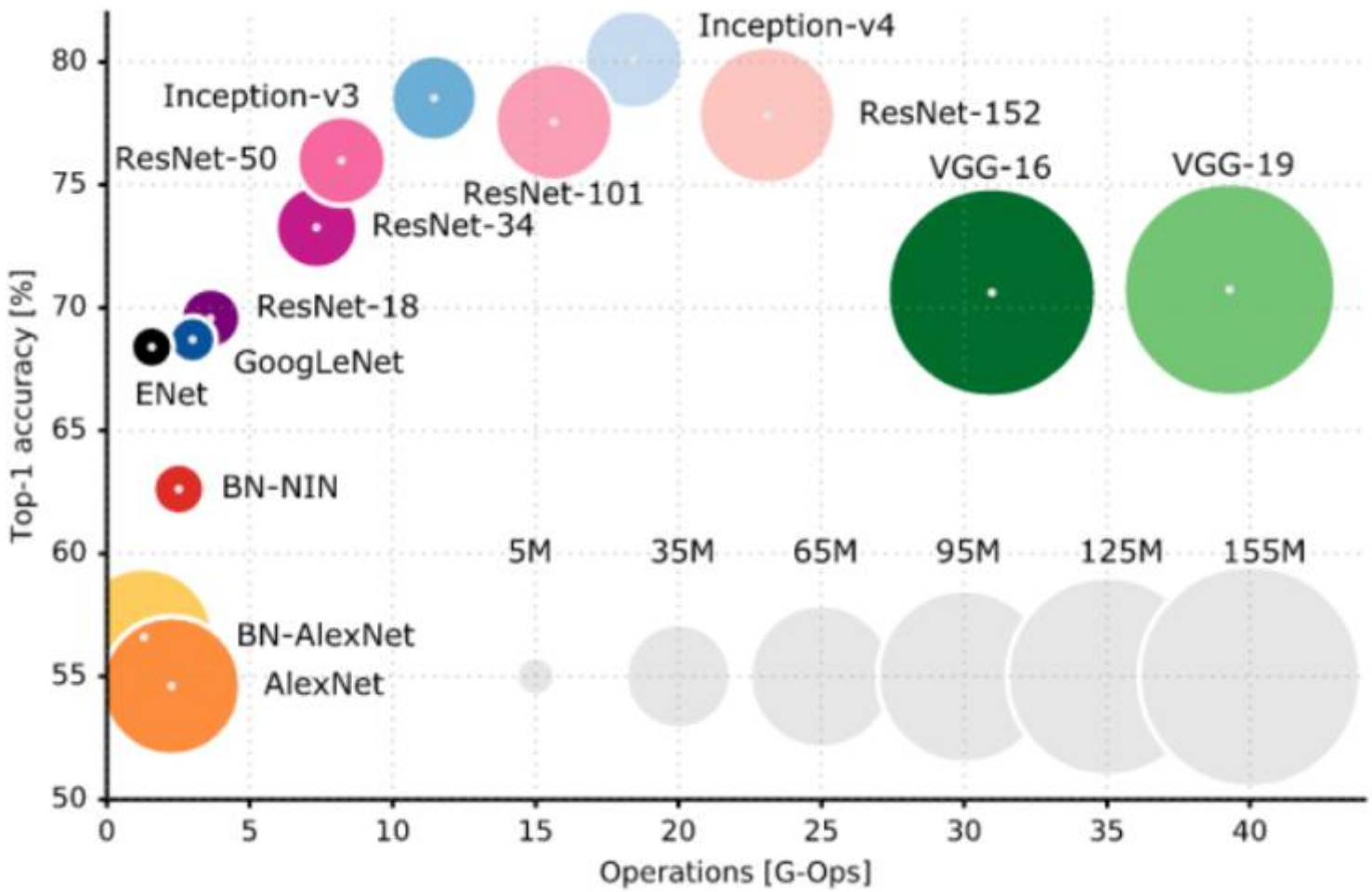
- ILSVRC 2014 winner
- Key feature: The Inception Module
  - Parallel paths with different receptive field sizes and operations
  - Capture sparse patterns of correlation in the feature map sets.

GoogLeNet (2015)



C. Szegedy et al., Going deeper with convolutions, CVPR 2015

Performance of Visual Recognition



https://culurciello.github.io/tech/2016/06/04/nets.html

References

Exercise

- Calculate the total number of parameters (with bias) required when a RGB image of 28 x 28 pixels
  - is input to a hidden layer consisting of 100 units
  - is convolved by 8 filters of 5 x 5 pixels each with padding = 2 and stride = 1
- Dropout is a regularization technique used to reduce overfitting in CNNs. It is usually applied to the hidden units of higher layers. During the training phase of a neural network, a fraction of the hidden units is randomly dropped at every iteration. Explain why overfitting can be reduced in this way.

Exercise Solution

*Ref: https://www.learnopencv.com/number-of-parameters-and-tensor-sizes-in-convolutional-neural-network/

(a) $(28x28x3+1)x100 = 2.353x10^5$

(b) $(5x5x3+1)x8 = 608$

Exercise Solution

*Ref: https://www.quora.com/What-does-a-dropout-in-neural-networks-mean

Dropout is a way to regularize the neural network. During training, it may happen that neurons of a particular layer may always become influenced only by the output of a particular neuron in the previous layer. In that case, the neural network would overfit. Dropout prevents overfitting and regularizes by randomly cutting the connections (also known as dropping the connection) between neurons in successive layers during training.